

# Day 2: Embedded systems, MicroPython, and sensor data collection

Naeemullah Khan

[naeemullah.khan@kaust.edu.sa](mailto:naeemullah.khan@kaust.edu.sa)



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology

KAUST Academy  
King Abdullah University of Science and Technology

October 2, 2026

- ▶ A model is only as good as its data. On an edge device, you make the data: the sensor, the sampling rate, and the labels are your decisions.
- ▶ To read a sensor, you must know the memory, the clock, and the buses of the microcontroller.
- ▶ In the lab you read the IMU of the XIAOML Kit with MicroPython and record a labelled motion dataset.
- ▶ Day 3 trains a model on this dataset. An error in the data today gives a wrong accuracy tomorrow.

After this day you can:

- ▶ Describe the memory, the clock, and the buses of a microcontroller.
- ▶ Select a sampling rate and a window length for a sensor signal.
- ▶ Write a MicroPython program that reads a sensor and sends the data.
- ▶ Build a labelled dataset with a correct split between training and test data.

## Part 1 Microcontroller anatomy

Exercise: find the RAM, the flash, and the sensor buses in a data sheet extract

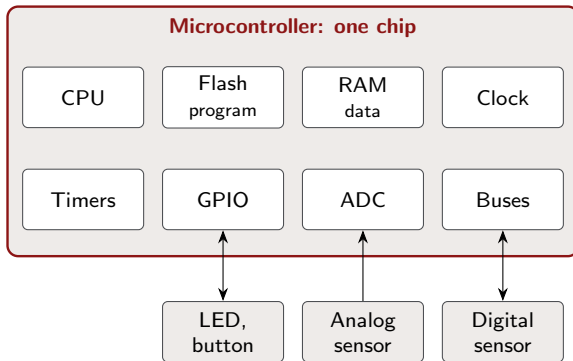
## Part 2 Sensors and sampling

Exercise: select the sampling rate and the window for three signals

## Part 3 Sensor data engineering

Exercise: find the leakage error in a dataset split

# A microcontroller is a computer on one chip



- ▶ The processor, the memory, and the peripherals are in one package.
- ▶ A peripheral is a circuit of the chip that connects the processor to the outside: a pin, a converter, a bus.
- ▶ The chip runs one program, the firmware. The program starts when the power comes on.

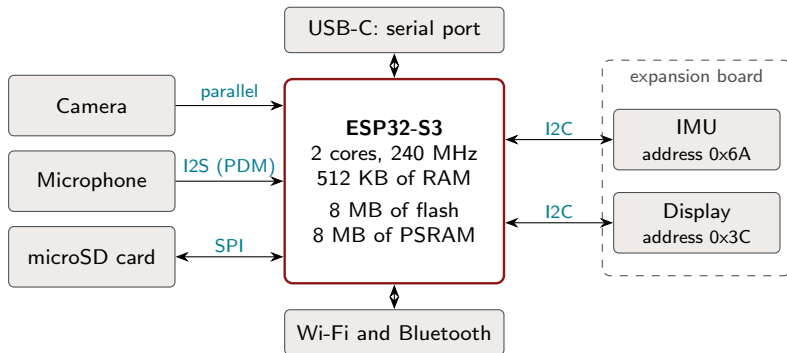
# A microcontroller is not a small Raspberry Pi

|          | <b>XIAO ESP32S3<br/>(microcontroller)</b> | <b>Raspberry Pi 5<br/>(single-board computer)</b> |
|----------|-------------------------------------------|---------------------------------------------------|
| Software | One program, the firmware                 | Linux and many processes                          |
| RAM      | 512 KB, and 8 MB of PSRAM                 | 8 GB                                              |
| Storage  | 8 MB of flash                             | microSD card                                      |
| Timing   | The program controls each microsecond     | The operating system can pause a program          |
| Pins     | Direct access to GPIO, ADC, and buses     | GPIO and buses through Linux. No ADC.             |
| Sleep    | Modes that use microamperes               | No such mode in normal use                        |

**Key point:** Use a microcontroller when the timing of a sensor and the energy decide. Use a single-board computer when the model needs gigabytes of memory.

Source: Kit pages of *Machine Learning Systems*.

# The parts of the XIAOML Kit and their buses



- ▶ The camera, the microphone, and the card slot are on the XIAO. The IMU and the display are on the expansion board.
- ▶ Each part uses the bus that fits its data rate. This part of the lecture explains each bus.

Source: *Machine Learning Systems*, XIAOML Kit, "Setup".

# Three types of memory

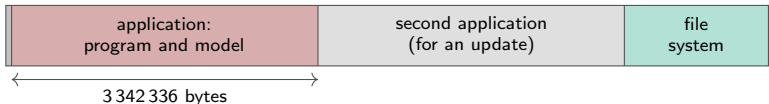
|               | Flash                                                | RAM                                                  | PSRAM                                      |
|---------------|------------------------------------------------------|------------------------------------------------------|--------------------------------------------|
| Holds         | The program, the constants, the weights of the model | The variables, the stack, the heap, the tensor arena | Large buffers: an image, a long audio clip |
| With no power | Keeps its content                                    | Loses its content                                    | Loses its content                          |
| Write         | Slow, in blocks                                      | Fast                                                 | Slower than RAM                            |
| On the XIAO   | 8 MB                                                 | 512 KB                                               | 8 MB                                       |

- ▶ A model uses two memories. The weights are constants in flash. The activations are in RAM.
- ▶ Day 1 gave the two checks: the model size against the flash, and the peak activation memory against the RAM.

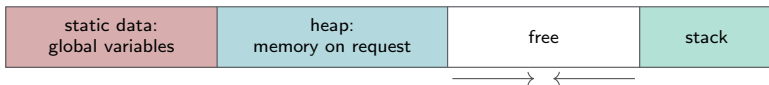
Source: Sizes: *Machine Learning Systems*, XIAO ML Kit, "Setup"; *ESP32-S3 Series Datasheet*, Espressif Systems.

# Where the program and the data are

## Flash of the XIAO: 8 MB



## RAM of the program



- ▶ The build gives the application one part of the flash. A model that is larger than this part does not fit.
- ▶ The heap and the stack grow into the free RAM. If they meet, the program stops with an error.

Source: Partition table default\_8MB of the Arduino core for the ESP32. The RAM bar shows the principle, not measured sizes.

- ▶ PSRAM is pseudo-static RAM: dynamic memory cells with a controller that refreshes them.
- ▶ It connects to the processor through a serial bus. An access needs more time than an access to the internal RAM.
- ▶ The program must ask for PSRAM. In Arduino, the function `ps_malloc()` takes memory from it.

## Use the PSRAM for

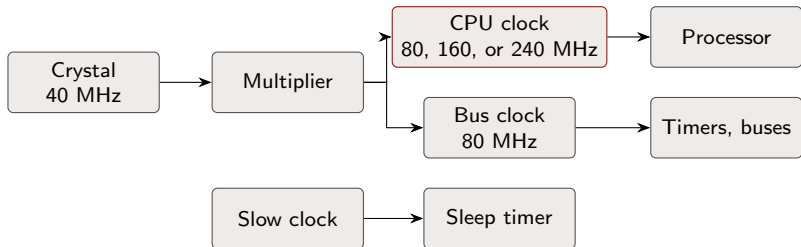
- ▶ an image buffer of the camera
- ▶ a tensor arena that does not fit the internal RAM

## Use the internal RAM for

- ▶ data that the program reads often
- ▶ a model that must be fast

**Key point:** The build option `Tools > PSRAM > OPI PSRAM` of Day 1 makes the 8 MB available. With the option off, the program has only the internal RAM.

# The clock sets the speed and the power



- ▶ The processor does one step at each clock cycle. A higher frequency gives more steps each second.
- ▶ A higher frequency also uses more power. A program that waits for a sensor can use a lower frequency.
- ▶ The timers count clock cycles. A constant sampling rate comes from a timer, not from the speed of the program.

Source: Clock values: *ESP32-S3 Series Datasheet*, Espressif Systems.

- ▶ A GPIO pin has two states: 0 V (low) and 3.3 V (high).
- ▶ As an output, the program sets the state: an LED, a relay.
- ▶ As an input, the program reads the state: a button, a motion detector.
- ▶ An open input has no defined state. A pull-up resistor holds it at high.
- ▶ The pins of the XIAO accept 3.3 V. A signal of 5 V damages a pin.

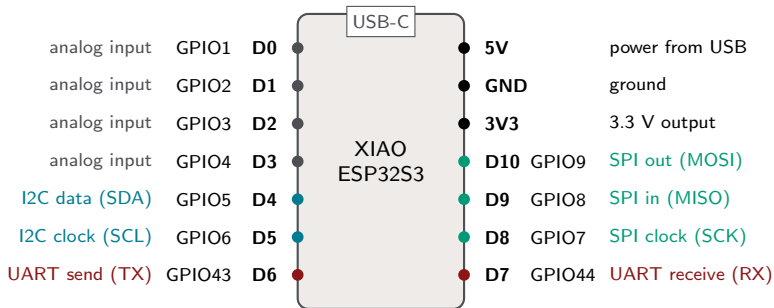
```
from machine import Pin

# The LED of the XIAO is on GPIO21.
# It is on when the pin is low.
led = Pin(21, Pin.OUT)
led.value(0)      # LED on
led.value(1)      # LED off

# The boot button is on GPIO0.
# It connects the pin to 0 V.
button = Pin(0, Pin.IN, Pin.PULL_UP)
print(button.value()) # 0 = pressed
```

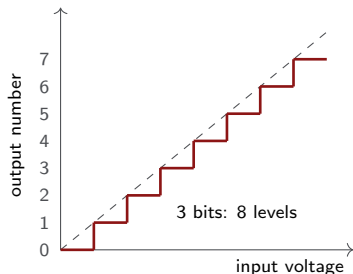
**Key point:** Read the pin number and the active level in the board documentation. The LED of the XIAO has inverted logic.

# The pins of the XIAO



- ▶ One pin can have more than one function. The program selects the function.
- ▶ The board uses other pins inside: GPIO21 for the LED, GPIO41 and GPIO42 for the microphone, and 14 pins for the camera.
- ▶ The expansion board connects the IMU and the display to D4 and D5.

Source: Pin numbers: file `pins_arduino.h` of the Arduino core for the ESP32, board XIA0\_ESP32S3.



- ▶ An analog-to-digital converter (ADC) measures a voltage and gives an integer.
- ▶ With  $n$  bits, the ADC has  $2^n$  levels.

$$\text{step} = \frac{V_{\text{range}}}{2^n}$$

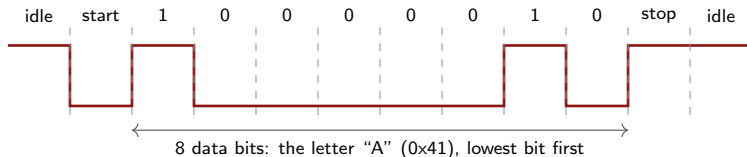
- ▶ The ADC of the ESP32-S3 has 12 bits: 4096 levels. For a range of 3.3 V, one step is 0.8 mV.
- ▶ A change that is smaller than one step is not visible.

**Key point:** The IMU and the microphone of the kit have their own ADC. They send numbers on a bus. The ADC of the XIAO is for analog sensors, for example a light sensor.

|      | Signal wires             | Devices                       | Speed                         | On the kit                |
|------|--------------------------|-------------------------------|-------------------------------|---------------------------|
| UART | 2: transmit, receive     | 2, point to point             | Low: 115 200 bit/s is typical | Serial port to the laptop |
| I2C  | 2: data, clock           | Many, each with an address    | Low: 100 or 400 kbit/s        | IMU, display              |
| SPI  | 4: clock, 2 data, select | Many, each with a select wire | High: megabits each second    | microSD card              |
| I2S  | 2 or 3: clocks, data     | 1 audio device                | Fixed by the audio rate       | Microphone                |

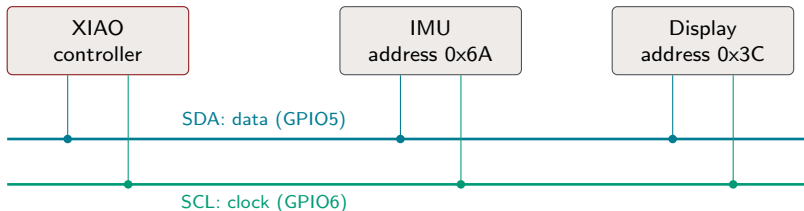
- ▶ “Serial” means that the bits go one after the other on one wire.
- ▶ More wires and a clock wire give more speed. Fewer wires use fewer pins.

# UART: two wires and no clock



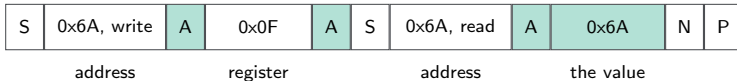
- ▶ One wire for each direction: TX of one device goes to RX of the other device.
- ▶ No clock wire. The two devices must use the same bit rate, the baud rate.
- ▶ One byte needs 10 bits on the wire. At 115 200 baud the limit is 11 520 bytes each second.
- ▶ The XIAO gives a serial port over USB. The MicroPython prompt and `print()` use this port.

# I2C: two wires and many devices



- ▶ The controller makes the clock and starts each transfer.
- ▶ Each target device has an address of 7 bits. The controller sends the address first. Only that device answers.
- ▶ Resistors pull the two wires to 3.3 V. A device can only pull a wire to 0 V.
- ▶ Typical speeds: 100 kbit/s and 400 kbit/s. This is enough for an IMU. It is too slow for audio or images.

# I2C: read one register of the IMU



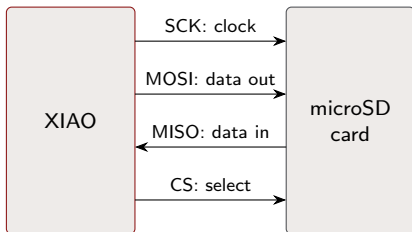
S = start, P = stop, A = acknowledge, N = no acknowledge. White: the controller sends. Green: the IMU sends.

- ▶ A sensor is a list of registers. Each register has a number and holds one byte.
- ▶ The register 0x0F of the IMU holds its identity. The value is 0x6A for the LSM6DS3TR-C.

```
from machine import I2C, Pin
i2c = I2C(0, sda=Pin(5), scl=Pin(6), freq=400000)
print([hex(a) for a in i2c.scan()])          # ['0x3c', '0x6a']
print(hex(i2c.readfrom_mem(0x6A, 0x0F, 1)[0])) # 0x6a
```

Source: Register values: library "Seeed Arduino LSM6DS3". The output is the expected output. Nobody measured it on the kit.

# SPI: four wires and more speed



- ▶ One clock wire and one data wire for each direction.
- ▶ No address. Each device has its own select wire (CS).
- ▶ Data goes in the two directions at the same time.
- ▶ The speed is megabits each second.
- ▶ Cost: one more pin for each device.

**Key point:** I2C saves pins. SPI gives speed. A sensor with a low data rate usually has I2C. A memory card or a fast display has SPI.

## Audio: I2S and PDM

- ▶ I2S sends audio samples at an exact rate. It has clock wires and one data wire.
- ▶ The microphone of the kit uses PDM: a stream of single bits at a high rate.
- ▶ A filter in the ESP32-S3 makes samples of 16 bits.
- ▶ Pins: clock GPIO42, data GPIO41.

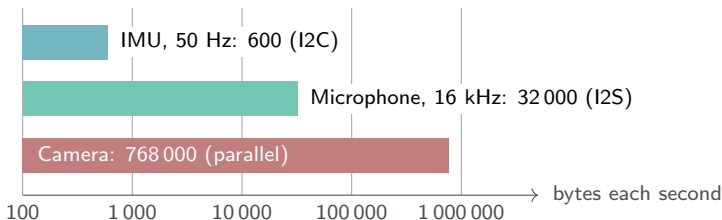
## Images: a parallel bus

- ▶ The camera sends 8 bits at the same time on 8 data pins.
- ▶ More signals mark each pixel, each line, and each image.
- ▶ The bus uses 14 pins. They are on the camera connector of the XIAO.

**Key point:** The hardware clocks the audio and the image data. The program reads complete buffers. It does not read single samples.

Source: Pin numbers: *Machine Learning Systems*, XIAO ML Kit, "Setup".

# The data rate selects the bus

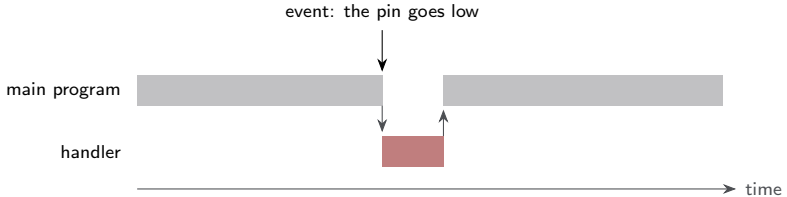


- ▶ IMU:  $6 \text{ values} \times 2 \text{ bytes} \times 50 \text{ samples each second} = 600 \text{ bytes each second}$ .
- ▶ Microphone:  $16\,000 \text{ samples} \times 2 \text{ bytes} = 32\,000 \text{ bytes each second}$ .
- ▶ Camera:  $320 \times 240 \text{ pixels} \times 1 \text{ byte} \times 10 \text{ images each second} = 768\,000 \text{ bytes each second}$ .

**Key point:** From the IMU to the camera, the data rate grows by a factor of 1280. The bus, the buffer, and the model must follow.

Source: The scale is logarithmic. The rates are the settings of the course labs. The camera rate of 10 images each second is an example.

# Interrupts: react to an event



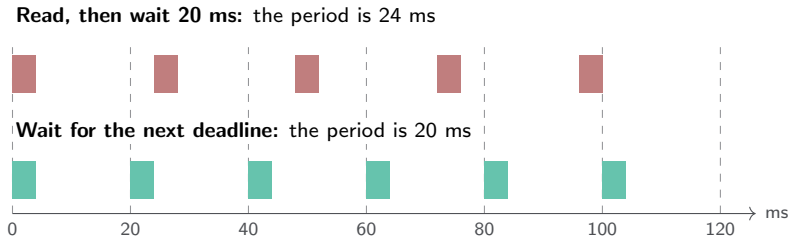
- ▶ Polling: the program reads the pin again and again. It can miss a short event.
- ▶ Interrupt: the hardware stops the main program and starts a handler function.
- ▶ Keep the handler short. Set a flag. Do the work in the main program.

```
from machine import Pin
pressed = False

def on_press(pin):
    global pressed
    pressed = True    # set a flag only

button = Pin(0, Pin.IN, Pin.PULL_UP)
button.irq(trigger=Pin.IRQ_FALLING,
            handler=on_press)
```

# Timers: a constant sampling rate



- ▶ The target is 50 Hz: one sample each 20 ms.
- ▶ A fixed wait after each read adds the read time to each period. With a read time of 4 ms, the rate is 41.7 Hz, not 50 Hz.
- ▶ A timer gives deadlines at 0, 20, 40 ms. The program waits for the next deadline. The read time does not change the rate.

**Key point:** The model of Day 3 expects 50 Hz. Data at a different rate is a different signal. Measure the rate in the lab.

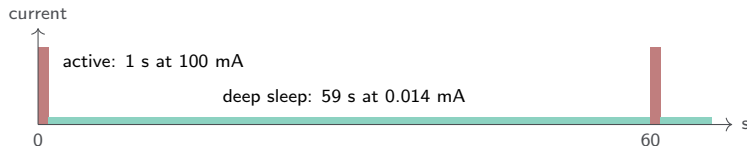
Source: The read time of 4 ms is an example, not a measured value.

| Mode        | State                                                             | After wake-up            | Current    |
|-------------|-------------------------------------------------------------------|--------------------------|------------|
| Active      | The processor and the Wi-Fi radio operate                         | not applicable           | 100 mA     |
| Modem sleep | The processor operates. The radio is off.                         | not applicable           | 27 mA      |
| Light sleep | The processor stops. The RAM keeps its content.                   | The program continues    | 2 mA       |
| Deep sleep  | Only a slow clock and a timer operate. The RAM loses its content. | The program starts again | 14 $\mu$ A |

- ▶ A timer or a pin wakes the board. In MicroPython: `machine.lightsleep(ms)` and `machine.deepsleep(ms)`.
- ▶ The values are for the XIAO ESP32S3 with no camera board, at 3.8 V. The board with the camera needs more.

Source: Current values: Seeed Studio wiki, "Getting Started with Seeed Studio XIAO ESP32S3 Series". Modes: ESP-IDF Programming Guide, "Sleep Modes".

# The duty cycle decides the battery life



$$I_{\text{mean}} = \frac{1 \times 100 \text{ mA} + 59 \times 0.014 \text{ mA}}{60} = 1.68 \text{ mA}$$

## Always active

Battery of 500 mAh:  $500/100 = 5$  hours.

## Active for 1 s each minute

$500/1.68 = 298$  hours, about 12 days.

**Key point:** The time in sleep decides the energy of an always-on sensor. Day 9 measures the energy of one inference.

Source: A calculation with the currents of the frame before. The battery size is an example. A real battery gives less.

An extract from the data sheet of a different TinyML board:

---

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| Microcontroller | nRF52840: Cortex-M4F, 64 MHz, 1 MB of flash, 256 KB of RAM |
| IMU             | BMI270 and BMM150 (9 axes), I2C                            |
| Microphone      | MP34DT06JTR, PDM output                                    |
| Pressure        | LPS22HB, I2C                                               |
| Humidity        | HS3003 (temperature and humidity), I2C                     |
| Light, gesture  | APDS-9960, I2C                                             |

---

1. How much flash and how much RAM does the board have?
2. Which bus does each sensor use?
3. Four sensors use the same two wires. How does the microcontroller select one of them?
4. A model has 300 KB of weights and a tensor arena of 100 KB. The program needs 300 KB of flash. Does the model fit? Which number do you need in addition?

Work with your neighbour for 5 minutes.

Source: Data sheet of the Arduino Nano 33 BLE Sense Rev2 (ABX00069).

1. 1 MB of flash and 256 KB of RAM. The board has no PSRAM.
2. The IMU, the pressure sensor, the temperature and humidity sensor, and the light sensor use I2C. The microphone uses PDM, which the microcontroller reads with its audio interface.
3. Each I2C device has its own address. The controller sends the address at the start of a transfer. Only that device answers.
4. **Flash:**  $300 + 300 = 600$  KB. This is less than 1 MB, so the weights fit.  
**RAM:** the arena needs 100 KB of the 256 KB. You also need the RAM that the program uses for its own data. The model fits if the program uses less than 156 KB.

The same four questions apply to each new board: the flash, the RAM, the sensors, and the bus of each sensor.

## Part 1 Microcontroller anatomy

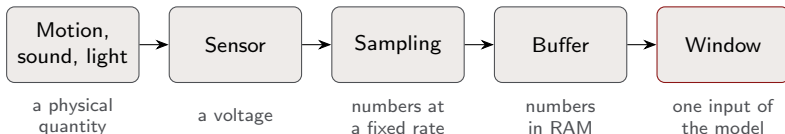
Exercise: find the RAM, the flash, and the sensor buses in a data sheet extract

## Part 2 Sensors and sampling

Exercise: select the sampling rate and the window for three signals

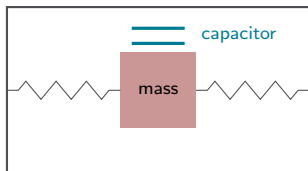
## Part 3 Sensor data engineering

Exercise: find the leakage error in a dataset split



- ▶ Each step has a setting that you select: the range of the sensor, the sampling rate, the size of the buffer, the length of the window.
- ▶ The model sees only the window. Information that a step removes does not come back.
- ▶ The same settings must apply when you collect the training data and when the device runs the model.

**Key point:** The data collection is part of the model design. You decide it before you train.



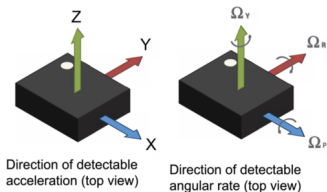
## Accelerometer

- ▶ A small mass on springs moves when the chip accelerates. The distance changes a capacitance.
- ▶ Unit:  $g$ . The sensor also measures gravity: a board at rest reads  $1\ g$  on one axis.

## Gyroscope

- ▶ It measures the rate of rotation around each axis.
- ▶ Unit: degrees each second (dps).

**Key point:** An IMU with 6 axes gives 6 numbers for each sample: three accelerations and three rotation rates.



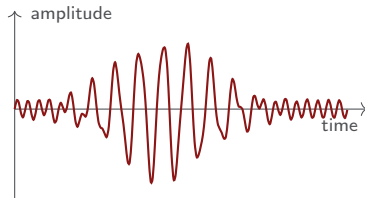
| Accelerometer range | One count |
|---------------------|-----------|
| $\pm 2 g$           | 0.061 mg  |
| $\pm 4 g$           | 0.122 mg  |
| $\pm 8 g$           | 0.244 mg  |
| $\pm 16 g$          | 0.488 mg  |

Each value has 16 bits: 65 536 levels.  
Gyroscope ranges: 125, 250, 500, 1000, and 2000 dps.

- ▶ A larger range gives a larger step. Select the smallest range that the motion does not exceed.
- ▶ The Arduino library of the kit sets 16 g and 2000 dps. The lab uses the same values, so Day 2 and Day 3 see the same numbers.

Source: Figure and ranges: *Machine Learning Systems*, XIAOML Kit, "Motion Classification and Anomaly Detection" (CC BY-NC-SA 4.0). Count values: library "Seeed Arduino LSM6DS3".

- ▶ Sound is a change of air pressure. A membrane in the microphone moves with the pressure.
- ▶ The microphone of the kit is digital. The XIAO receives samples of 16 bits.
- ▶ The labs use 16 000 samples each second.
- ▶ One second of audio is 16 000 samples, or 32 000 bytes.



a word: less than 1 second

**Key point:** Audio has one channel and a high rate. An IMU has six channels and a low rate. The window of a keyword model holds 16 000 numbers. The window of a motion model holds some hundreds.

Source: Audio settings: *Machine Learning Systems*, XIAOML Kit, “Keyword Spotting”. The curve is a drawing, not a recording.

- ▶ An image sensor is a grid of light sensors: the pixels.
- ▶ The resolution is the sampling in space. The frame rate is the sampling in time.
- ▶ The pixel format sets the bytes for each pixel.
- ▶ A large image needs the PSRAM. The model input is much smaller than the sensor image.

| Image       | Format            | Bytes     |
|-------------|-------------------|-----------|
| 96 × 96     | grayscale, 1 byte | 9 216     |
| 96 × 96     | RGB888, 3 bytes   | 27 648    |
| 320 × 240   | grayscale, 1 byte | 76 800    |
| 320 × 240   | RGB565, 2 bytes   | 153 600   |
| 1600 × 1200 | RGB565, 2 bytes   | 3 840 000 |

Bytes = width × height × bytes for each pixel.

**Key point:** The largest image of the table is 3.7 MB. It is seven times the internal RAM of the XIAO. The camera can also send JPEG data, which is smaller.

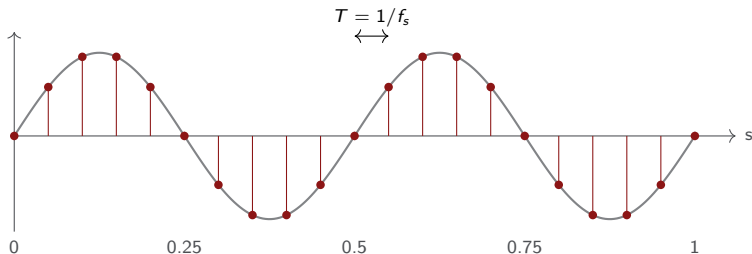
# Three sensors, three types of signal

|                      | IMU                          | Microphone                 | Camera                  |
|----------------------|------------------------------|----------------------------|-------------------------|
| Signal               | Time series, 6 channels      | Time series, 1 channel     | Image, 2 dimensions     |
| Rate in the labs     | 50 samples each second       | 16 000 samples each second | Some images each second |
| One window           | 2 s                          | 1 s                        | 1 image                 |
| Values in one window | $2 \times 50 \times 6 = 600$ | 16 000                     | $96 \times 96 = 9216$   |
| Day of the course    | 2, 3, 4                      | 5                          | 5                       |

- ▶ For each sensor you select a rate and a window. The next frames give the rules.
- ▶ The motion lab of the kit uses only the 3 accelerometer channels: 300 values in one window.

Source: Settings of the kit labs: *Machine Learning Systems*, XIAOML Kit chapters.

# Sampling: a continuous signal becomes numbers



- ▶ The sampling rate  $f_s$  is the number of samples each second. The period  $T$  is the time between two samples.
- ▶ The figure shows a signal of 2 Hz with  $f_s = 20$  Hz: 10 samples for each cycle.
- ▶ Between two samples, the system knows nothing about the signal.

**Key point:** Sampling makes the signal discrete in time. Quantization makes it discrete in value. The two steps lose information.

## Rule of Nyquist and Shannon

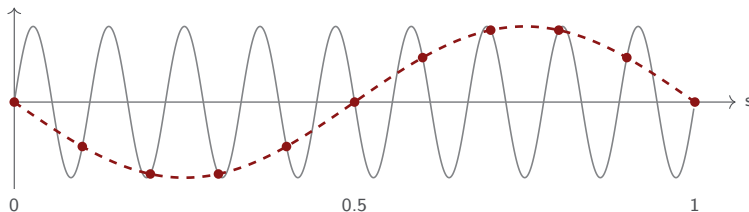
A sampled signal keeps all information only if the sampling rate is more than two times the highest frequency of the signal.

$$f_s > 2 f_{\max}$$

- ▶ The frequency  $f_s/2$  is the Nyquist frequency.
- ▶ With  $f_s = 50$  Hz, the system can represent frequencies below 25 Hz.
- ▶ The rule gives the minimum. With exactly two samples for each cycle, the amplitude is not reliable.
- ▶ In practice, use a rate with a margin above the minimum.

**Key point:** First find the highest frequency that the application needs. Then select the sampling rate.

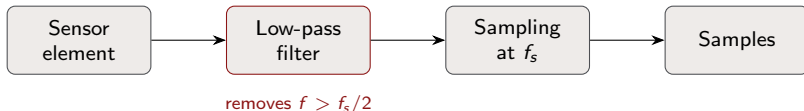
# Aliasing: a frequency that is not there



- ▶ Gray: the real signal of 9 Hz. Red points: the samples at  $f_s = 10$  Hz.
- ▶ The samples are on a curve of 1 Hz. The system sees a signal of 1 Hz.
- ▶ A frequency  $f$  above  $f_s/2$  appears as the alias  $|f - kf_s|$ , for the integer  $k$  that gives a result below  $f_s/2$ . Here:  $|9 - 10| = 1$  Hz.

**Key point:** The data show a wrong frequency and give no warning. A model then learns from a signal that does not exist.

# Aliasing: remove the high frequencies before sampling



- ▶ A filter after the sampling cannot remove an alias. The alias has the same samples as a real low frequency.
- ▶ A digital sensor has this filter inside. The filter fits the output data rate of the sensor.
- ▶ If the program reads the sensor at a lower rate than its output data rate, the filter does not protect the program.

## In the lab

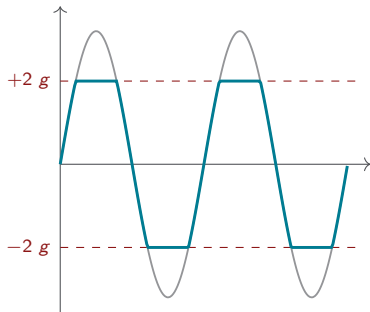
The IMU makes 416 samples each second. The program reads 50 of them. This is correct for hand motion, which is slow. It is not correct for the vibration of a motor.

# How to select the sampling rate

| Signal                          | Frequencies that matter           | Minimum rate                  | Typical rate   |
|---------------------------------|-----------------------------------|-------------------------------|----------------|
| Motion of a person or a package | Some hertz                        | Some tens of hertz            | 50 to 100 Hz   |
| Speech                          | Up to about 4 kHz for a telephone | 8 kHz                         | 16 kHz         |
| Vibration of a machine          | Up to some kilohertz              | Two times the fault frequency | Some kilohertz |

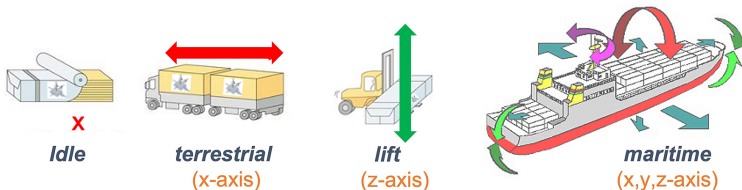
1. Find the highest frequency that the application needs.
2. Multiply by two, and add a margin.
3. Check the cost: a higher rate needs more RAM, more computation, and more energy.
4. Check that the sensor, the bus, and the program reach the rate.

Source: Motion and speech rates: *Machine Learning Systems*, XIAOML Kit chapters. Telephone band: ITU-T Recommendation G.711.



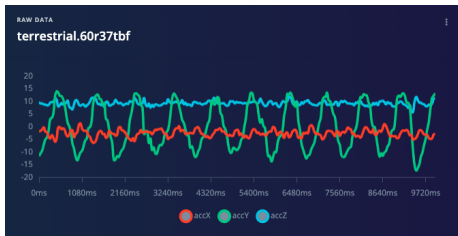
- ▶ Gray: the real acceleration, up to  $3.2\text{ g}$ .
  - ▶ Blue: the output of a sensor with a range of  $\pm 2\text{ g}$ .
  - ▶ The sensor cuts each value outside its range. This is clipping.
  - ▶ A clipped signal has a different form and new frequencies.
- 
- ▶ A range that is too small clips. A range that is too large loses resolution.
  - ▶ Look at the raw data before you train. A flat top at the limit of the range shows clipping.

# The motion classes of the lab



- ▶ The project of the kit lab: a sensor on a package finds how the package moves.
- ▶ Four classes: **idle** (no motion), **terrestrial** (horizontal motion), **lift** (vertical motion), and **maritime** (motion on all axes).
- ▶ You make each motion with the kit in your hand.

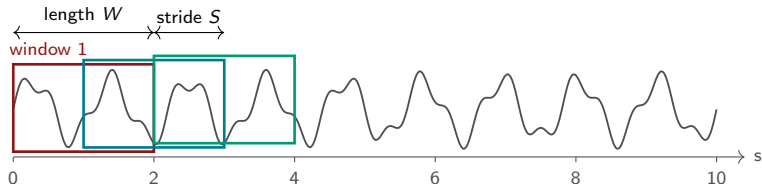
Source: Figure: *Machine Learning Systems*, XIAOML Kit, "Motion Classification and Anomaly Detection" (CC BY-NC-SA 4.0).



- ▶ One recording of the class “terrestrial”: 10 s at 50 Hz, three axes, in  $\text{m/s}^2$ .
- ▶ A model takes a window of a fixed length, not the complete recording.
- ▶ The window must hold the pattern. One cycle of this motion needs about 1 s. A window of 2 s holds two cycles.

**Key point:** A window of 2 s at 50 Hz with 3 axes holds  $2 \times 50 \times 3 = 300$  values. This is the input size of the Day 3 model.

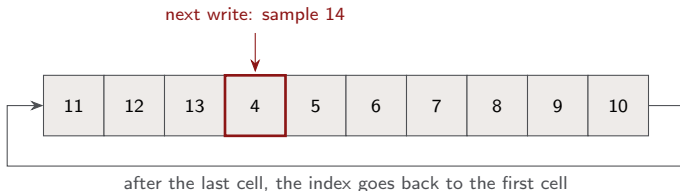
Source: Figure and window: *Machine Learning Systems*, XIAOML Kit, “Motion Classification and Anomaly Detection” (CC BY-NC-SA 4.0).



$$\text{number of windows} = \left\lfloor \frac{L - W}{S} \right\rfloor + 1 \quad (L: \text{length of the recording})$$

- ▶ The kit lab:  $L = 10$  s,  $W = 2$  s,  $S = 0.2$  s gives  $(10 - 2)/0.2 + 1 = 41$  windows.
- ▶ A longer window holds more of the pattern. It needs more RAM, and the first result comes later.
- ▶ A smaller stride gives more windows. With  $S = 0.2$  s, two adjacent windows share 90 percent of their samples.

Source: The figure shows a stride of 1 s. Window and stride of the example: *Machine Learning Systems*, XIAOML Kit, “Motion Classification and Anomaly Detection”.



- ▶ A ring buffer holds the newest  $N$  samples. A new sample replaces the oldest sample.
- ▶ One part of the program writes at the sampling rate. A different part reads a window when the model is ready.
- ▶ If the reader is too slow, the writer overwrites samples that the reader did not use.

## Size of the buffer

Motion:  $2 \text{ s} \times 50 \times 3 \text{ values} \times 4 \text{ bytes} = 1200 \text{ bytes}$ . Audio:  $1 \text{ s} \times 16\,000 \text{ values} \times 2 \text{ bytes} = 32\,000 \text{ bytes}$ .

```
from array import array

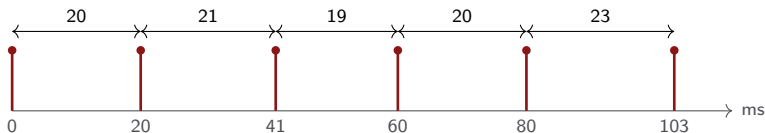
N = 100                                # 2 s at 50 Hz
ax = array("f", [0.0] * N)           # allocate once
head = 0                               # next write

def push(value):
    global head
    ax[head] = value
    head = (head + 1) % N

def window():
    # oldest sample first
    return [ax[(head + i) % N]
            for i in range(N)]
```

- ▶ `array("f")` holds 4 bytes for each value. A list of Python numbers needs more.
- ▶ The program allocates the buffer one time, before the loop.
- ▶ `push()` allocates no memory. It is safe in the sampling loop.
- ▶ `window()` makes a new list. Call it outside the sampling loop.

**Key point:** MicroPython has a garbage collector. An allocation in the sampling loop can start it, and then one period is longer.



- ▶ The board writes a time stamp with each sample. The period is the difference between two time stamps.
- ▶ The mean period gives the real rate. The variation of the period is the jitter.
- ▶ Use the time stamp of the board. The arrival time on the laptop is not constant, because USB sends the lines in groups.

## The check of the lab

The mean rate is within 1 percent of 50 Hz. At most 1 percent of the periods are outside 18 to 22 ms. No line is lost.

Source: The periods of the figure are an example, not measured values.

|            | MicroPython                                       | C++ (Arduino)                                      |
|------------|---------------------------------------------------|----------------------------------------------------|
| Work cycle | You enter a line at the prompt and see the result | You edit, compile, and upload the complete program |
| Speed      | An interpreter runs the code: slower              | The compiler makes machine code: faster            |
| Memory     | The interpreter and its heap use RAM              | The program uses only the memory that it needs     |
| Timing     | The garbage collector can pause the program       | The program controls the timing                    |
| ML runtime | Not in the standard firmware                      | TensorFlow Lite Micro, Edge Impulse library        |

## Use MicroPython to

learn a sensor, test a bus, and collect data at a low rate.

## Use C++ to

run a model, read audio or images, and build the final product.

## MicroPython (today)

```
from machine import I2C, Pin
from lsm6ds3 import LSM6DS3

i2c = I2C(0, sda=Pin(5), scl=Pin(6),
          freq=400000)
imu = LSM6DS3(i2c)

ax, ay, az, gx, gy, gz = imu.read()
print(ax, ay, az)
```

## C++ with Arduino (Day 3)

```
#include <LSM6DS3.h>
#include <Wire.h>
LSM6DS3 myIMU(I2C_MODE, 0x6A);

void setup() {
  Serial.begin(115200);
  myIMU.begin();
}

void loop() {
  float ax = myIMU.readFloatAccelX();
  Serial.println(ax);
}
```

- ▶ The two programs use the same bus, the same address, and the same registers.
- ▶ The course uses MicroPython today. Days 3 to 5 use C++, because the model runs with TensorFlow Lite Micro.

Source: C++ code: *Machine Learning Systems*, XIAOML Kit, "Motion Classification and Anomaly Detection" (short form).

For each signal, select a sampling rate and a window length. Then calculate the number of values in one window.

1. **Gesture.** An accelerometer with 3 axes on a wrist. The motion has frequencies up to 15 Hz. One gesture needs about 1.5 s.
2. **Speech.** One microphone. The model needs the frequencies up to 6 kHz. One keyword needs less than 1 s.
3. **Vibration.** An accelerometer with 1 axis on a motor. A damaged bearing makes a vibration at 1.2 kHz. The motor turns 25 times each second. The model must see 5 turns.

Use these rates: 50 Hz, 100 Hz, 4 kHz, 8 kHz, 16 kHz.

Work with your neighbour for 8 minutes.

|           | Minimum | Rate   | Window | Values in one window         |
|-----------|---------|--------|--------|------------------------------|
| Gesture   | 30 Hz   | 50 Hz  | 2 s    | $2 \times 50 \times 3 = 300$ |
| Speech    | 12 kHz  | 16 kHz | 1 s    | 16 000                       |
| Vibration | 2.4 kHz | 4 kHz  | 0.2 s  | $0.2 \times 4000 = 800$      |

- ▶ Gesture: 100 Hz is also correct. It gives 600 values for the same information.
- ▶ Speech: 8 kHz is too low. The frequencies between 4 and 6 kHz then give aliases.
- ▶ Vibration: 5 turns need  $5/25 = 0.2$  s. A rate of 4 kHz needs C++. A MicroPython loop with one bus transfer for each sample is probably too slow. Measure before you decide.

## Part 1 Microcontroller anatomy

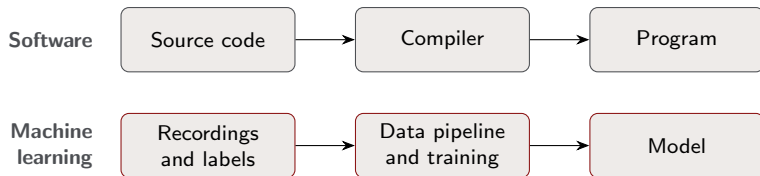
Exercise: find the RAM, the flash, and the sensor buses in a data sheet extract

## Part 2 Sensors and sampling

Exercise: select the sampling rate and the window for three signals

## Part 3 Sensor data engineering

Exercise: find the leakage error in a dataset split



- ▶ A change of the data changes the model, as a change of the source code changes the program.
- ▶ So the data needs the same methods as code: a written procedure, tests, versions, and a review.
- ▶ On an edge device you make the data with the sensor. No public dataset has the signals of your device in your application.

**Key point:** To remove one recording is to remove one line of code. To train again is to compile again.

# A data error gives no error message

- ▶ A code error stops the program. A data error does not.
- ▶ The error moves through each later stage: the features, the training, the test, the deployment.
- ▶ The model trains. The accuracy looks correct. The device fails in use.

| Code error       | Data error     |
|------------------|----------------|
| Compiler message | No message     |
| Program stops    | Model is worse |
| You debug code   | You debug data |

## Examples for a motion sensor

The sampling rate was 42 Hz, not 50 Hz. One class has the wrong label. One person recorded all data of one class.

Source: The effect has the name “data cascade”: Sambasivan et al., “Everyone wants to do the model work, not the data work”, CHI 2021.



- ▶ The board sends samples. A Python program on the laptop stores them in files.
- ▶ Each file is one recording of one class.
- ▶ You split the files into a training set and a test set. Day 3 cuts the windows and trains the model.
- ▶ You also upload the same files to Edge Impulse Studio and inspect them there.

A collection protocol is a written procedure. You write it before the first recording, and each person follows it.

| Item                     | The protocol of the lab                                              |
|--------------------------|----------------------------------------------------------------------|
| Classes                  | idle, terrestrial, lift, maritime                                    |
| Definition of each class | One sentence for each motion: the axis and the speed                 |
| Sensor settings          | 50 Hz, range 16 g, three accelerometer axes and three gyroscope axes |
| Position of the kit      | In the hand, with the display up                                     |
| One recording            | 10 s of one motion with no pause                                     |
| Quantity                 | About 2 minutes for each class: 12 recordings                        |
| Sessions                 | A minimum of three sessions for each class                           |
| File name                | label, session, number: <code>lift.s2.07.csv</code>                  |

Source: Classes, rate, recording length, and quantity: *Machine Learning Systems*, XIAOML Kit, "Motion Classification and Anomaly Detection".

# Collect the variation that the device will see

| Factor      | Change it between sessions            |
|-------------|---------------------------------------|
| Person      | Each student of the group records     |
| Speed       | Slow, medium, and fast motion         |
| Amplitude   | Small and large motion                |
| Orientation | Small changes of the angle of the kit |

- ▶ A model learns each difference between the classes, including the differences that you did not intend.
- ▶ If one person records all “lift” data, the model can learn the person, not the motion.
- ▶ If the kit is flat only in the class “idle”, the model can learn the orientation.

**Key point:** Change in the data all factors that change in the use of the device. Keep constant only what stays constant in use.

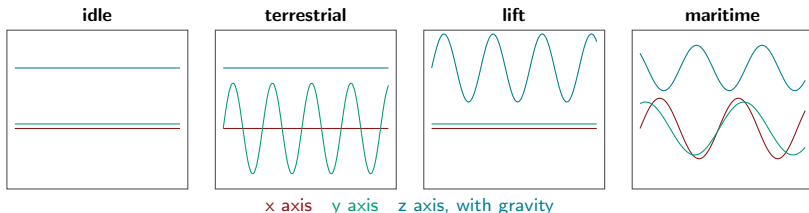
## Rules

- ▶ Give the label when you record, not later.
- ▶ One recording has one label.
- ▶ Write the definition of each class. Two persons must give the same label to the same motion.
- ▶ Store the label in the file name and in the file.
- ▶ Start the motion first. Then start the recording.
- ▶ A label error in the training set makes the model worse. A label error in the test set makes the result wrong.

## Typical label errors

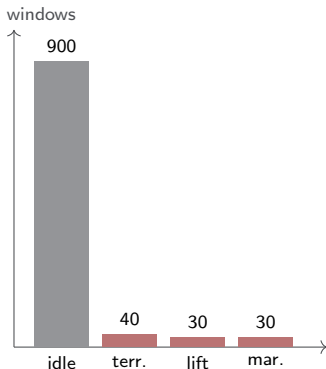
- ▶ The first second of a recording holds the start of the motion, or no motion.
- ▶ A recording has the label of the recording before it.
- ▶ Two classes have no clear limit: a slow “maritime” motion and a “terrestrial” motion.

# Look at each recording



- ▶ Make a plot of each recording immediately after you record it.
- ▶ Check that the plot agrees with the label: the correct axis moves, and the motion fills the complete recording.
- ▶ Delete a wrong recording and record it again. This costs 10 seconds now. It costs hours after the training.

Source: The curves are drawings of the expected form, not recordings.



- ▶ A package is at rest most of the time. A log of real use has almost only “idle”.
- ▶ A model that always answers “idle” is correct for 900 of 1000 windows: an accuracy of 90 percent.
- ▶ This model finds no motion. The accuracy hides that.

- ▶ Record the same duration for each class.
- ▶ Report the result for each class, not only the total accuracy. Day 3 uses a confusion matrix.

Source: The numbers are an example for the calculation.

# Training set, validation set, and test set



| Set        | Use                                                   | Rule                        |
|------------|-------------------------------------------------------|-----------------------------|
| Training   | The optimizer fits the weights to this data.          | The largest set             |
| Validation | You select the model and its settings with this data. | Not used to fit the weights |
| Test       | You estimate the accuracy on new data.                | Used one time, at the end   |

**Key point:** The test set must be independent of the training set. If it is not, the test accuracy says nothing about the device in use.

Source: The percentages are a typical example.

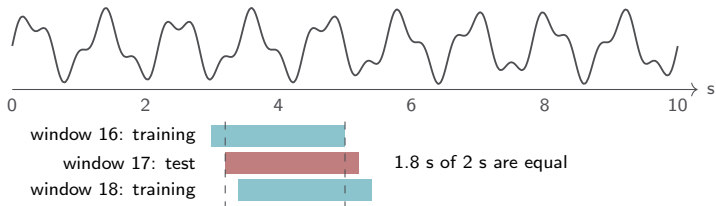
## Definition

Data leakage: information from the test data reaches the training. The test result is then better than the result on new data.

| Type                | How it occurs                                                                     |
|---------------------|-----------------------------------------------------------------------------------|
| Overlapping windows | Two windows share samples. One is in the training set, the other in the test set. |
| The same recording  | Windows of one recording are in the two sets.                                     |
| The same session    | Recordings of one person in one position are in the two sets.                     |
| Preprocessing       | The mean and the standard deviation for the normalization come from all data.     |
| Copies              | The same file is in the dataset two times.                                        |

**Key point:** Leakage gives no error. The only sign is a test accuracy that is much better than the accuracy on the device.

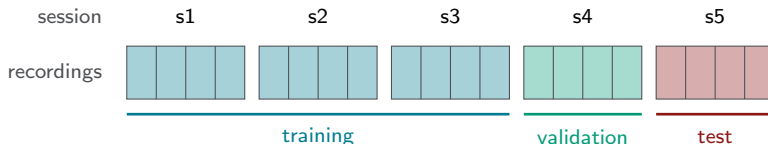
# Leakage between overlapping windows



- ▶ Window length 2 s, stride 0.2 s: adjacent windows share 90 percent of their samples.
- ▶ A random split of the windows puts window 17 in the test set and its neighbours in the training set.
- ▶ The model saw almost all samples of the test window during the training. The test measures memory.

**Key point:** Do not split windows at random. Split the recordings first. Then cut the windows.

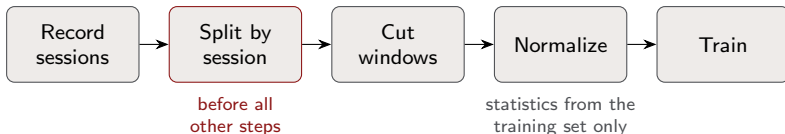
# Split by recording session



- ▶ A session is one period of recording with one person, one position of the kit, and no long pause.
- ▶ Recordings of one session are similar: the same hand, the same speed, the same angle.
- ▶ All recordings of one session go to the same set.
- ▶ A stronger test: the test session comes from a person who has no recording in the training set.

**Key point:** The check of the lab: no session is in the training set and also in the test set.

# The correct order of the steps



- ▶ **Split first.** Each later step then sees only its own set.
- ▶ **Cut the windows in each set.** An overlap inside the training set is not leakage. It gives more training examples.
- ▶ **Normalize with the training set.** Calculate the mean and the standard deviation from the training set. Apply the same two numbers to the validation set, to the test set, and on the device.

**Key point:** A wrong order is the usual cause of leakage: first windows, then a random split.

A dataset changes: new recordings, corrected labels, a new split. Each change gives a different model.

## One version holds

- ▶ the files
- ▶ the protocol and the data card
- ▶ the list of the sessions in each set
- ▶ a hash value of each file

## Rules

- ▶ Give each version a number: v1, v2.
- ▶ Do not change a version after a model used it. Make a new version.
- ▶ Write the dataset version next to each trained model.
- ▶ Keep the raw recordings. Make the windows with a script.

**Key point:** A small dataset can be in git. For a large dataset, git holds only the list of the files and their hash values.

A data card is a short document that describes a dataset. It is the deliverable of the lab.

| Field            | Example                                                       |
|------------------|---------------------------------------------------------------|
| Name and version | Motion of a package, group 7, v1                              |
| Classes          | idle, terrestrial, lift, maritime, with one sentence for each |
| Sensor           | LSM6DS3TR-C, 50 Hz, range 16 <i>g</i> and 2000 dps            |
| Duration         | 120 s for each class, 480 s in total                          |
| Sessions         | 3 sessions, 2 persons                                         |
| Split            | Training: s1 and s2. Test: s3.                                |
| Checks           | Mean rate, lost samples, clipped samples                      |
| Known limits     | Two persons only. The kit was always in the hand.             |

- The field “known limits” says where the model can fail.

Source: The idea of a document for each dataset: Gebru et al., “Datasheets for Datasets”, 2021.  
The values are an example.

## Structure checks

Does each file have the correct form?

- ▶ the correct columns
- ▶ numbers in each cell
- ▶ the expected number of rows: 500 for 10 s at 50 Hz
- ▶ a sample counter with no gap

- ▶ A file can pass each structure check and still hold wrong data. A recording in  $\text{m/s}^2$  in a dataset in  $g$  has the correct form.
- ▶ Run the checks with a script after each session, not at the end.

## Statistical checks

Do the values make sense?

- ▶ the mean rate and the jitter
- ▶ no value at the limit of the range (clipping)
- ▶ no constant signal in a motion class
- ▶ at rest, the length of the acceleration vector is near  $1 g$

**Key point:** The structure check finds a broken file. The statistical check finds a wrong measurement.

# The same settings for training and for the device

| Setting       | Data collection (Day 2)        | Risk on the device (Day 3)               |
|---------------|--------------------------------|------------------------------------------|
| Sampling rate | 50 Hz                          | A loop with a different period           |
| Sensor range  | 16 g, 2000 dps                 | A library with a different default value |
| Unit          | $g$                            | $m/s^2$ : a factor of 9.81               |
| Axes          | x, y, z of the kit             | A kit in a different position            |
| Window        | 2 s                            | A buffer of a different length           |
| Normalization | Statistics of the training set | Different numbers, or none               |

- ▶ A difference between the training data and the device data has the name training-serving skew. The accuracy decreases, and no error appears.

**Key point:** Write the settings in the data card. Day 3 reads them from there.

- ▶ Edge Impulse Studio is a cloud tool for TinyML projects: data, features, training, and a library for the device.
- ▶ In Part D of the lab you upload the CSV files of your dataset.
- ▶ The Studio reads the label from the file name: the text before the first full stop.
- ▶ You inspect each recording as a plot, and you see the duration of each class.

## Check the split

- ▶ The Studio can split the files into training and test data automatically.
- ▶ This split works on files. It does not know your sessions.
- ▶ Upload the test session as test data yourself.

**Key point:** A tool makes the collection fast. The protocol, the labels, and the split stay your decisions.

Source: *Machine Learning Systems*, XIAOML Kit, “Motion Classification and Anomaly Detection”; Edge Impulse documentation.

# Exercise: find the leakage error

A team builds a motion classifier with four classes.

- ▶ Three persons (A, B, C) record each class one time for 60 s.
- ▶ The team cuts windows of 2 s with a stride of 0.2 s. It normalizes with the mean and the standard deviation of all windows.
- ▶ The team mixes all windows: 80 percent for the training set, 20 percent for the test set.
- ▶ The test accuracy is 99 percent. On the device, with person D, the model is correct for 7 of 10 motions.

1. Find two leakage errors.
2. How many windows does one recording give? Which part of a test window is also in a training window?
3. Write a correct procedure with the same recordings.

Work with your neighbour for 8 minutes. The scenario and its numbers are an example.

1. **Error 1:** the team split the windows at random. Overlapping windows of one recording are in the two sets.  
**Error 2:** the normalization uses the mean and the standard deviation of all windows, with the test windows.
2. One recording gives  $(60 - 2)/0.2 + 1 = 291$  windows. A test window shares 1.8 s of its 2 s with an adjacent window: 90 percent. With 80 percent of the windows in the training set, an adjacent window is almost always there.
3. A correct procedure:
  - Split by person: A and B for the training, C for the test.
  - Cut the windows in each set.
  - Calculate the mean and the standard deviation from the windows of A and B. Apply them to C and on the device.

The test accuracy then decreases. It is nearer to the result with person D, because C is also a new person for the model.

- ▶ A microcontroller has flash for the program, RAM for the data, and buses for the sensors: I2C, SPI, I2S, and UART.
- ▶ The sampling rate must be more than two times the highest frequency of the signal. A lower rate gives aliasing.
- ▶ A window is the input of the model. Its length and its overlap are design decisions.
- ▶ Overlapping windows of one recording are almost equal. Split the data by recording session, not by window.

**Key point:** Fix the sampling rate, the labels, and the split before you train. A model cannot repair an error in its data.

## Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 4 “Data Engineering”.
- ▶ *Machine Learning Systems*, hardware kits, XIAOML Kit: “Setup” and “Motion Classification and Anomaly Detection”.
- ▶ *XIAO: Big Power, Small Board*, chapter 2.4 “Rhythmic Dance with a Triaxial Accelerometer”.

## Data sheets

- ▶ “ESP32-S3 Series Datasheet”, Espressif Systems.
- ▶ “LSM6DS3TR-C: iNEMO inertial module”, STMicroelectronics.

## Tools

- ▶ MicroPython documentation: <https://docs.micropython.org/>
- ▶ Edge Impulse documentation: <https://docs.edgeimpulse.com/>

This lecture adapts material from these sources:

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. [mlsysbook.ai](https://mlsysbook.ai), CC BY-NC-SA 4.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. [github.com/Mjrovai/XIAO-ESP32S3-Sense](https://github.com/Mjrovai/XIAO-ESP32S3-Sense), Apache-2.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai. [github.com/Mjrovai/XIAO\\_Big\\_Power\\_Small\\_Board-ebook](https://github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook), GPL-3.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.